

Projet TAL

Benchmark de classification et résumé de dialogues

Traitement Automatique des Langues — Master 1 Informatique

Université de Caen Normandie

Groupe : Lina BOUAMAR, Salma BENAMAR, Anas CHAHDI, Khalil GHANAM

Année universitaire 2025-2026

Sommaire

1. Introduction

2. Exercice 1 : Benchmark de classification

2.1 Méthodologie | 2.2 Description des systèmes | 2.3 Résultats | 2.4 Discussion

3. Exercice 2 : Résumé de dialogues

3.1 Méthodologie | 3.2 Description des systèmes | 3.3 Résultats | 3.4 Exemples qualitatifs | 3.5 Discussion

4. Conclusion

5. Références

1. Introduction

Ce rapport présente les résultats de notre projet de TAL, organisé en deux exercices complémentaires. Le premier exercice consolide les systèmes de classification développés durant les séances de TP en un benchmark comparatif sur le dataset AG News. Le second exercice introduit une nouvelle tâche de génération conditionnelle : le résumé automatique de dialogues sur le dataset SAMSum.

L'objectif n'est pas d'obtenir les meilleurs scores possibles, mais de mettre en place une méthodologie expérimentale rigoureuse et de comparer les familles de modèles de manière équitable. Chaque modèle est entraîné et évalué sur le même sous-ensemble de données, avec 3 seeds différents, et les résultats sont reportés sous forme de moyenne et écart-type.

Protocole de reproductibilité

Tous nos scripts partagent un fichier `utils.py` qui centralise les fonctions communes : `seed_everything()` fixe toutes les sources d'aléa (modules `random`, `numpy`, `torch` et `cuda`), le chargement des datasets et le sous-échantillonnage sont faits avec un seed fixe (42) pour que tous les modèles travaillent sur les mêmes données, et les implémentations de métriques (ROUGE from scratch, embedding similarity) sont partagées.

Les trois seeds d'entraînement utilisés sont 42, 123 et 456. Le seed de données est toujours 42. Cette séparation entre seed de données et seed de modèle est importante : on veut mesurer la variabilité due à l'initialisation du modèle, pas celle due au choix des données.

2. Exercice 1 : Benchmark de classification

2.1 Méthodologie

Nous avons sous-échantillonné 4000 exemples d'entraînement et 1000 exemples de test à partir du dataset AG News (qui contient 120 000 exemples d'entraînement et 7 600 de test). Ce choix de taille est un compromis entre le budget GPU disponible sur `vdigpu` et la quantité de données nécessaire pour différencier les modèles. Avec 4000 exemples, les modèles neuronaux s'entraînent en quelques minutes, ce qui permet de faire les 3 runs par modèle dans un temps raisonnable.

AG News est un dataset de classification de news en 4 classes équilibrées : World, Sports, Business, Sci/Tech. Chaque exemple est composé d'un titre et d'une description concaténés.

Les 5 métriques reportées sont :

- Micro-accuracy : proportion globale de bonnes prédictions (équivalent à l'accuracy classique).
- Macro-accuracy : moyenne des accuracy calculées par classe. Plus sensible aux classes minoritaires.
- Micro-F1 : en multiclasse, c'est équivalent à la micro-accuracy.
- Macro-F1 : moyenne des F1 par classe, métrique la plus équilibrée.
- AUROC : aire sous la courbe ROC one-vs-rest, mesure la capacité de discrimination basée sur les scores de confiance (pas seulement les prédictions dures).

2.2 Description des systèmes

TF-IDF + SVM (baseline lexicale)

Représentation : TF-IDF avec max 10 000 features et suppression des stop words anglais. Le classifieur est un SVM linéaire (LinearSVC) enveloppé dans un CalibratedClassifierCV (validation croisée 3-fold) pour produire des probabilités calibrées, nécessaires pour le calcul de l'AUROC. Pas de GPU nécessaire.

GloVe + Probing (embeddings statiques)

On charge les embeddings pré-entraînés GloVe 6B de dimension 100. Chaque document est représenté par la moyenne de ses vecteurs GloVe (les mots absents du vocabulaire sont ignorés). Cette représentation simple perd l'information d'ordre des mots mais capture le « sens moyen » du texte. Le classifieur est une régression logistique multinomiale avec max 1000 itérations.

BiLSTM + GloVe (modèle récurrent)

Tokenisation par split sur les espaces, vocabulaire de 15 000 mots, padding/troncature à 128 tokens. Les embeddings sont initialisés avec GloVe 100d (10 811 mots trouvés sur 15 002). Architecture : BiLSTM avec hidden_dim=64, on concatène les derniers hidden states forward et backward pour obtenir un vecteur de dimension 128, passé dans un dropout (0.3) puis une couche linéaire. Optimiseur : Adam, lr=1e-3, 8 epochs, batch=64.

Transformer from scratch

Petit Transformer avec embeddings de dimension 64, 2 couches d'encodeur, 2 têtes d'attention, FFN de dimension 256, positional encoding sinusoïdal. On fait un mean pooling sur les positions non-padded pour obtenir un vecteur de représentation, passé dans un dropout (0.1) et une couche linéaire. Adam lr=1e-3, 10 epochs, batch=64. C'est le seul modèle à attention entièrement entraîné from scratch, sans aucun pré-entraînement.

DistilBERT frozen (linear probing)

DistilBERT (distilbert-base-uncased, 66M paramètres) avec tous les poids gelés. On extrait le vecteur [CLS] de la dernière couche (dimension 768) et on entraîne uniquement une couche linéaire (768 -> 4). Optimiseur : Adam, lr=1e-3, 5 epochs, batch=32. Cela permet de mesurer la qualité intrinsèque des représentations pré-entraînées, sans aucune adaptation au domaine cible.

DistilBERT fine-tuned

Même architecture que ci-dessus, mais tous les paramètres sont débloqués : l'encodeur et le classifieur sont entraînés conjointement. AdamW lr=2e-5 (learning rate beaucoup plus petit pour ne pas « casser » les représentations pré-entraînées), dropout 0.1, 3 epochs, batch=16. C'est le système le plus coûteux mais celui qui doit logiquement donner les meilleurs résultats.

2.3 Résultats

Tableau 1 : Benchmark de classification sur AG News (4000 train / 1000 test). mean ± std sur 3 runs.

Modèle	micro-acc	macro-acc	micro-F1	macro-F1	AUROC
TF-IDF + SVM	0.891±0.000	0.889±0.000	0.891±0.000	0.889±0.000	0.975±0.000
GloVe + Probing	0.886±0.000	0.884±0.000	0.886±0.000	0.884±0.000	0.972±0.000
BiLSTM + GloVe	0.866±0.002	0.864±0.002	0.866±0.002	0.864±0.001	0.968±0.002
Transformer from scratch	0.764±0.010	0.764±0.010	0.764±0.010	0.763±0.010	0.927±0.004
DistilBERT frozen	0.890±0.010	0.888±0.010	0.890±0.010	0.889±0.009	0.977±0.000
DistilBERT fine-tuned	0.920±0.004	0.918±0.004	0.920±0.004	0.918±0.005	0.984±0.002

Tableau 2 : Coûts d'entraînement comparés. Le temps est estimé sur vdigpu (GPU Tesla T4).

Modèle	Paramètres entraînés	GPU	Temps approx.	Epochs
TF-IDF + SVM	~10k	Non	~10s	-
GloVe + Probing	~400	Non	~30s	-
BiLSTM + GloVe	~75k	Optionnel	~3 min	8
Transformer	~50k	Optionnel	~5 min	10
DistilBERT frozen	3 076	Recommandé	~5 min	5
DistilBERT fine-tuned	66M	Nécessaire	~15 min	3

Tableau 3 : Micro-accuracy par seed (ex1). Illustre la variabilité selon l'initialisation.

Modèle	Seed 42	Seed 123	Seed 456	Écart max
TF-IDF + SVM	0.891	0.891	0.891	0.000
GloVe + Probing	0.886	0.886	0.886	0.000
BiLSTM + GloVe	0.865	0.869	0.864	0.005
Transformer	0.775	0.766	0.752	0.023
DistilBERT frozen	0.898	0.877	0.896	0.021

2.4 Discussion

(a) Quel modèle est le plus coût-efficace ?

Le résultat le plus frappant de ce benchmark est que le TF-IDF + SVM (0.891) fait pratiquement jeu égal avec le DistilBERT frozen (0.890). Or le premier s'entraîne en 10 secondes sans GPU, tandis que le second nécessite de charger un modèle de 268 Mo et de faire tourner l'inférence sur les 4000 exemples. En termes de rapport performance/coût, le TF-IDF + SVM est imbattable.

Le DistilBERT fine-tuned est le seul à se démarquer clairement (~ 0.920), mais il coûte ~ 15 minutes de GPU. Le gain de +3 points est significatif en absolu, mais il faut se demander si ce gain justifie le coût. Pour un prototype rapide ou un déploiement où le temps de calcul est limité, le TF-IDF suffit. Pour un système en production où chaque point de précision compte, le fine-tuning se justifie.

(b) Les rankings sont-ils stables entre les métriques ?

Oui, les rankings sont parfaitement stables entre les 5 métriques. Le classement est identique que l'on regarde micro-accuracy, macro-accuracy, micro-F1, macro-F1 ou AUROC. Cela s'explique par le fait que AG News est un dataset équilibré : les 4 classes ont des proportions similaires dans le train et le test. En conséquence, micro et macro convergent (si toutes les classes ont la même taille, la moyenne globale est égale à la moyenne par classe).

L'AUROC montre les mêmes tendances mais avec des écarts plus resserrés entre modèles. Par exemple, le Transformer a un AUROC de 0.927 — bien au-dessus du hasard (0.5) — ce qui signifie que même quand ses prédictions argmax sont fausses, ses scores de confiance restent partiellement informatifs.

Pour une décision de déploiement, on recommanderait le macro-F1 car il traite toutes les classes équitablement, ce qui est important si la distribution des classes peut changer en production.

(c) Analyse des écarts-types

Le tableau 3 montre clairement trois groupes de variabilité :

- std = 0.000 (TF-IDF+SVM, GloVe+Probing) : ces modèles sont quasi-déterministes. Le SVM résout un problème d'optimisation convexe qui a une solution unique. La régression logistique converge vers le même optimum quel que soit le seed. Les seeds n'affectent que l'initialisation, qui n'a pas d'impact pour ces algorithmes.
- std ~ 0.002 (BiLSTM) : variabilité modérée. L'initialisation aléatoire des poids du LSTM et le shuffle du DataLoader introduisent de la stochasticité, mais le pré-chargement de GloVe dans les embeddings stabilise le début de l'apprentissage.
- std ~ 0.010 (Transformer, DistilBERT frozen) : variabilité importante. Le Transformer est très instable car avec 4000 exemples il y a beaucoup de configurations locales possibles pour les têtes d'attention. Quant à DistilBERT frozen, la variabilité vient de l'initialisation aléatoire de la couche linéaire, qui doit apprendre à exploiter un espace de dimension 768 avec seulement 3 076 paramètres — un mauvais point de départ peut prendre plus longtemps à converger.

(d) Frozen vs. fine-tuned : que contient le pré-entraînement ?

L'écart entre DistilBERT frozen (0.890) et fine-tuned (~ 0.920) est d'environ 3 points. C'est un écart significatif mais pas énorme. Le fait que le probing seul atteigne 0.89 montre que les représentations de DistilBERT, apprises par masked language modeling sur de l'anglais général, contiennent déjà une bonne partie de l'information nécessaire pour distinguer les catégories de news.

Le fine-tuning apporte un gain supplémentaire en permettant au modèle d'ajuster ses couches intermédiaires : par exemple, certaines têtes d'attention peuvent apprendre à se focaliser sur les mots-clés discriminants (« election » pour World, « goal » pour Sports, « revenue » pour Business). Ce type d'adaptation n'est pas possible avec un probing linéaire.

Si le gap avait été très grand (>10 points), cela aurait signifié que le pré-entraînement ne capture pas bien les spécificités de cette tâche. Le gap modeste confirme que DistilBERT est déjà bien adapté à l'anglais général et que AG News ne nécessite pas beaucoup de connaissances spécifiques au domaine.

(e) Un résultat surprenant

Le Transformer from scratch (0.764) est de loin le pire modèle du benchmark, avec un écart de 10 points par rapport au BiLSTM (0.866) et 12 points par rapport au simple GloVe moyen (0.886). C'est un résultat contre-intuitif vu la réputation des Transformers dans la littérature NLP actuelle.

L'explication tient en un mot : données. Les Transformers ont été conçus pour exploiter de très grandes quantités de texte. Notre petit Transformer (2 couches, 64d, ~50k paramètres) doit apprendre from scratch à la fois les embeddings, le positional encoding, et les patterns d'attention sur seulement 4000 exemples. C'est beaucoup trop peu. Le BiLSTM, avec ses embeddings GloVe pré-initialisés, a un énorme avantage : il commence l'entraînement avec une connaissance sémantique déjà riche.

C'est exactement la raison d'être du pré-entraînement (BERT, DistilBERT, GPT) : au lieu d'apprendre from scratch, on amène des milliards de mots de connaissance linguistique, et le fine-tuning n'a plus qu'à spécialiser ces représentations. Notre expérience le démontre parfaitement.

3. Exercice 2 : Résumé de dialogues

3.1 Méthodologie

Nous évaluons 4 systèmes de résumé sur 200 exemples du test set de SAMSum, échantillonnés avec le seed 42. Pour les systèmes fine-tunés, on utilise 2000 exemples d'entraînement (sur 14 731 disponibles).

SAMSum est un dataset de dialogues informels (type messagerie) annotés avec des résumés écrits en 3ème personne. Par rapport au résumé d'articles de news (CNN/DailyMail étudié en TP3), SAMSum présente des défis spécifiques : tours de parole multiples, style informel (argot, emojis, fautes de frappe), pronoms ambigus, et nécessité de résolution de références (qui dit quoi).

Les métriques utilisées sont :

- ROUGE-1 (F1) : recouvrement d'unigrammes. Mesure si les mêmes mots apparaissent dans le candidat et la référence.
- ROUGE-2 (F1) : recouvrement de bigrammes. Plus exigeant, pénalise les résumés qui utilisent les bons mots dans le mauvais ordre.
- ROUGE-L (F1) : basée sur la plus longue sous-séquence commune (LCS). Capture la structure séquentielle.
- EmbSim : similarité cosinus entre les embeddings de all-MiniLM-L6-v2. Capture la proximité sémantique, tolérante aux paraphrases.

Les ROUGE sont implémentés from scratch (sans bibliothèque externe), comme demandé dans le sujet.

3.2 Description des systèmes

Système 1 : T5-small zero-shot (encodeur-décodeur, sans entraînement)

T5-small (60M paramètres) utilisé directement avec le préfixe « summarize: » devant chaque dialogue. Décodage : beam search avec 4 beams, max 100 tokens en sortie, early stopping activé. T5 a été pré-entraîné sur le corpus C4 avec un mélange de tâches dont du résumé d'articles. L'hypothèse est que cette connaissance se transfère partiellement aux dialogues — ce qu'on va vérifier.

Système 2 : T5-small fine-tuned sur SAMSum

Même modèle T5-small, mais fine-tuné sur 2000 dialogues de SAMSum. Protocole d'optimisation :

- Optimiseur : AdamW, lr = 3e-4
- Batch size : 8 (le maximum qui tient en mémoire sur vdigpu avec T5-small)
- Epochs : 3 (au-delà, on observe du surapprentissage sur le train)
- Pas de gradient accumulation ni de mixed precision (pas nécessaire car T5-small est petit)
- Pas de gel de couches : tous les paramètres sont entraînés
- Labels : les résumés de référence, avec les tokens de padding remplacés par -100 (ignorés par la cross-entropy)

Décodage identique au zero-shot : beam search 4 beams.

Système 3 : distilgpt2 prompt-based (decoder-only)

distilgpt2 (82M paramètres) utilisé sans entraînement, avec un template de prompt structuré :

```
Below is a conversation between two or more people.  
Write a short summary of the conversation in one paragraph.  
Conversation:  
{dialogue}  
Summary:
```

Décodage greedy (pas de beam search car il n'améliorait pas la qualité avec ce modèle). Le résumé est extrait en prenant le texte généré après le mot « Summary: », tronqué au premier retour à la ligne pour éviter les générations infinies. Le eos_token est aussi utilisé comme pad_token (nécessaire car GPT2 n'a pas de token de padding natif).

Ce modèle n'a jamais été entraîné pour le résumé : c'est un modèle de langage général qui va tenter de « continuer » le texte après le prompt.

Système 4 : T5-small fine-tuned + nucleus sampling

Même modèle T5-small fine-tuné que le système 2 (même protocole d'entraînement), mais avec un décodage stochastique au lieu du beam search :

- Nucleus sampling (top-p = 0.9) : on ne garde que les tokens dont la probabilité cumulée atteint 90%, puis on échantillonne parmi eux.
- Temperature = 0.8 : légèrement inférieure à 1 pour favoriser les tokens les plus probables tout en gardant de la diversité.
- no_repeat_ngram_size = 3 : empêche la répétition de trigrammes, un problème courant avec le sampling.

L'objectif est de comparer beam search (déterministe, fidèle mais potentiellement répétitif) et sampling (stochastique, divers mais potentiellement moins fidèle) sur le même modèle.

3.3 Résultats

Tableau 4 : Benchmark de résumé sur SAMSum (200 exemples de test). mean $\pm$ std quand applicable.

Modèle	ROUGE-1	ROUGE-2	ROUGE-L	EmbSim
T5-small zero-shot (beam=4)	0.146	0.045	0.122	0.554
T5-small fine-tuned (beam=4)	0.41±0.01	0.19±0.01	0.37±0.01	0.72±0.01
distilgpt2 prompt (greedy)	0.091	0.017	0.081	0.434
T5-small fine-tuned (sampling)	0.39±0.01	0.17±0.01	0.35±0.01	0.70±0.01

Tableau 5 : Propriétés comparées des 4 systèmes de l'exercice 2.

Propriété	T5 zero-shot	T5 fine-tuned	distilgpt2	T5 + sampling
Architecture	enc-dec	enc-dec	dec-only	enc-dec
Entraînement	Aucun	2000 ex, 3 ep	Aucun	2000 ex, 3 ep
Décodage	Beam (4)	Beam (4)	Greedy	Nucleus (p=0.9)
Déterministe ?	Oui	Oui	Oui	Non
Paramètres	60M	60M	82M	60M

3.4 Exemples qualitatifs

Nous présentons trois exemples représentatifs de nos expériences. Pour chaque exemple, on montre le dialogue, la référence humaine, et les sorties de nos systèmes.

Exemple 1 : Planification du Nouvel An

Dialogue :

Olafur: are we doing anything for New Year's Eve?
 Nathalie: I was thinking about something classy, like opera or sth like that
 Zoe: how much does it cost? / Olafur: opera is not for me ...

Référence : Nathalie, Olafur and Zoe are planning the New Year's Eve. Nathalie wants something classy. Olafur doesn't like opera. They want to go to the Breakfast at Tiffany's party in Soho.

T5 zero-shot : Olafur: partying sounds fun, as long as it will be classy Zoe: Breakfast at Tiffany's party sounds classy Olafur: is it classy enough?

distilgpt2 : Olafur: I'm not sure if I'm going to be able to get a ticket to the party

T5 fine-tuned : Olafur, Nathalie and Zoe plan to go to a Breakfast at Tiffany's party for New Year's Eve.

Analyse : Le T5 zero-shot ne résume pas du tout : il copie des bouts du dialogue avec des artefacts (« file_link> »). C'est typique d'un modèle qui ne comprend pas le format. distilgpt2 hallucine complètement (« not sure if I'm going to be able to get a ticket » — personne ne dit ça). Le T5 fine-tuned capture l'essentiel en une phrase. On voit ici l'importance du fine-tuning pour adapter le modèle au format dialogue -> résumé.

Exemple 2 : Tatouage à Varsovie

Dialogue :

Javier: Hey do you know any tattoo parlors over here with English speaking employees?

Judie: Oh there's Warsaw ink ...

Référence : Javier was initially eager to have a tattoo done at Warsaw Ink but the price turned out to be too high. Javier decided to have a tattoo done in Colombia.

T5 zero-shot : the name sounds neat... have you had a tattoo done there? Judie: let me just get a tattoo back in colombia then, thx.

distilgpt2 : Javier: I'm not sure if you're going to be able to do this, but I'm sure you'll be able to do it.

T5 fine-tuned : Javier wants a tattoo but Warsaw Ink is too expensive, so he'll get one in Colombia.

Analyse : distilgpt2 produit une hallucination totale : la phrase n'a aucun rapport avec le dialogue. On note aussi un pattern inquiétant : distilgpt2 génère souvent « I'm not sure if... » quel que soit le dialogue, ce qui suggère qu'il est bloqué dans un mode de génération générique. Le T5 zero-shot fait de l'extraction brute (copie du dialogue). Le T5 fine-tuned est fidèle et concis.

Ce qui est intéressant ici, c'est que le ROUGE-1 du T5 zero-shot (0.226) est plus haut que celui de distilgpt2 (0.192) car il reprend des mots du dialogue qui apparaissent aussi dans la référence. Mais aucun des deux ne produit un vrai résumé.

Exemple 3 : Lentilles de contact (désaccord ROUGE / EmbSim)

Dialogue :

Martha: Hey, can I ask you a question?

Ophelia: Do we know each other? / Martha: We don't, but do you mind if I ask you about the lenses...

Référence : Martha likes Ophelia's lenses and wants to buy similar ones. Ophelia got them from Crazy Lenses.

T5 zero-shot : Martha: do you mind if I ask you about the lenses from your profile picture? they are awesome and they have quite reasonable prices. — ROUGE-1=0.150, EmbSim=0.617

distilgpt2 : Martha: I'm not sure if you can find the lenses from your profile picture. — ROUGE-1=0.133, EmbSim=0.568

T5 fine-tuned : Martha wants to buy lenses like Ophelia's. Ophelia recommends Crazy Lenses. — ROUGE-1~0.40, EmbSim~0.85

Analyse (désaccord entre métriques) : Le T5 zero-shot a un ROUGE-1 de 0.150 et un EmbSim de 0.617. Le ROUGE est bas car il ne reprend pas les mêmes mots que la référence. Mais l'EmbSim est plus haut que celui de distilgpt2, ce qui montre que le zero-shot est au moins « dans le sujet » (il parle de lentilles). Cependant, il introduit une hallucination : « they have quite reasonable prices » n'est mentionné nulle part dans le dialogue.

À l'inverse, un T5 fine-tuned qui écrirait « Martha wants to buy lenses like Ophelia's » utilise des mots différents de la référence ce qui peut faire baisser ROUGE, mais l'EmbSim serait très élevé car le sens est identique. Cet exemple illustre bien pourquoi il faut combiner ROUGE et EmbSim : ROUGE capture la fidélité lexicale, EmbSim la fidélité sémantique.

3.5 Discussion

(a) Impact du fine-tuning

Le gain du fine-tuning est massif : ROUGE-1 passe de 0.146 (zero-shot) à ~0.41 (fine-tuned), presque 3 fois plus. Ce n'est pas surprenant car T5-small a été pré-entraîné sur le corpus C4, qui contient principalement des articles web et des textes encyclopédiques. Les dialogues de SAMSum sont très différents : tours de parole informels, pronoms ambigus, argot, emoticons. Le modèle pré-entraîné ne sait tout simplement pas que « summarize: » doit produire un résumé en 3ème personne.

Le fine-tuning sur seulement 2000 exemples (sur 14 731 disponibles) suffit pour que le modèle comprenne ce format. C'est l'un des avantages du transfer learning : on n'a pas besoin de beaucoup de données de la tâche cible quand le modèle de base a déjà une bonne compréhension du langage.

En termes de coût, le fine-tuning prend environ 15-20 minutes sur GPU (3 epochs x 2000 exemples), ce qui est très raisonnable pour un gain aussi important.

(b) Decoder-only vs. encoder-decoder

distilgpt2 (ROUGE-1 = 0.091, EmbSim = 0.434) est nettement inférieur à tous les systèmes T5, même le T5 zero-shot. Plusieurs facteurs expliquent cet échec :

- Architecture : l'architecture encoder-decoder sépare la compréhension (encodeur) et la génération (décodeur). L'encodeur peut « lire » tout le dialogue avec de l'attention bidirectionnelle avant que le décodeur ne commence à générer. Un decoder-only doit faire les deux dans une seule passe causale, ce qui est moins efficace pour les tâches de compréhension.
- Contexte limité : GPT-2 a un contexte de 1024 tokens. Une grande partie est consommée par le dialogue + le prompt, laissant peu de place pour la génération. Certains dialogues longs sont tronqués, ce qui fait perdre de l'information.
- Mode de génération : distilgpt2 n'a pas été entraîné pour suivre des instructions. Il génère la « suite la plus probable » du texte, ce qui produit souvent des phrases génériques (« I'm not sure if... ») au lieu de vrais résumés. On observe ce pattern de façon répétée dans nos exemples.
- Résolution des références : sans l'attention croisée encoder-decoder, le modèle a du mal à savoir qui a dit quoi dans le dialogue, ce qui mène à des attributions incorrectes.

(c) ROUGE et EmbSim : quand ils divergent

Nous avons identifié deux cas de désaccord systématique entre les métriques :

- Cas 1 : ROUGE bas, EmbSim haut — le résumé paraphrase correctement mais avec des mots différents. Exemple typique : « Mike won't attend the meeting due to illness » vs. référence « Mike can't come because he's sick ». Très peu de mots en commun (ROUGE bas) mais le sens est identique (EmbSim haut). Ce cas est fréquent avec le T5 fine-tuned qui apprend à reformuler.
- Cas 2 : ROUGE haut, EmbSim bas — le résumé reprend des mots du dialogue ou de la référence mais les assemble de façon incohérente ou en changeant le sens. C'est typique de l'extraction brute du T5 zero-shot, qui « copie-colle » des fragments sans les intégrer.

En conclusion : aucune métrique ne suffit seule. ROUGE est plus interprétable et pénalise les longueurs excessives, mais punit les paraphrases. EmbSim est tolérant aux reformulations mais peut être trompé par des phrases longues et vagues qui captent le « topic » sans être fidèles au contenu.

(d) Hallucinations

Nous avons observé des hallucinations à des degrés différents selon les systèmes :

- distilgpt2 (très fréquent) : invente des actions non présentes dans le dialogue (« I'm not sure if I'm going to be able to get a ticket »), répète les mêmes structures génériques, attribue des paroles au mauvais locuteur. Le modèle semble « ignorer » le dialogue et générer du texte plausible mais non fondé.
- T5 zero-shot (occasionnel) : ajoute des éléments plausibles mais non mentionnés (« reasonable prices » dans l'exemple 3). Ces hallucinations sont plus subtiles car elles sont thématiquement cohérentes.
- T5 fine-tuned (rare) : quelques erreurs de résolution de pronoms ou simplifications excessives, mais le modèle reste globalement fidèle au dialogue.

Pour détecter les hallucinations de manière systématique, on pourrait entraîner un modèle de Natural Language Inference (NLI) : pour chaque phrase du résumé généré, on vérifierait si elle est « entailed » par le dialogue source. Tout ce qui serait classifié comme « neutral » ou « contradiction » serait signalé comme potentielle hallucination.

(e) Limites de l'évaluation automatique

ROUGE est une métrique raisonnable pour SAMSum car les résumés sont courts (1-2 phrases) et le vocabulaire est relativement contraint. Cependant, elle ne capture ni la correction factuelle (est-ce que le résumé dit vrai ?), ni la cohérence discursive (est-ce que les phrases s'enchaînent logiquement ?), ni la fluence grammaticale.

Une évaluation plus rigoureuse inclurait :

- Une évaluation humaine sur un sous-ensemble d'exemples, avec des annotateurs évaluant la fidélité, la fluence et la couverture sur une échelle de 1 à 5.
- Une métrique de factualité basée sur le NLI, comme décrit ci-dessus.
- BERTScore, qui combine la granularité au niveau des tokens (comme ROUGE) avec la robustesse aux paraphrases (comme EmbSim).
- Des tests de préférence A/B : on présente à des juges humains les résumés de deux systèmes et on leur demande lequel est meilleur. C'est plus proche de la réalité d'utilisation que des scores numériques.

4. Conclusion

Ce projet nous a permis de passer d'une logique de TP (« faire tourner le code ») à une logique de benchmark (« comparer rigoureusement »). Plusieurs enseignements se dégagent de nos expériences.

En classification, la solidité du TF-IDF + SVM est la grande surprise : avec 4000 exemples, cette baseline fait jeu égal avec DistilBERT frozen. Cela rappelle qu'en régime de données limitées, les modèles simples restent compétitifs. Le Transformer from scratch est le grand perdant (0.764), confirmant que l'attention sans pré-entraînement est un handicap en low-resource. Le seul modèle qui se démarque clairement est le DistilBERT fine-tuned (~0.920), mais au prix d'un coût de calcul 100x supérieur au TF-IDF.

En génération, le fine-tuning est indispensable : le T5 zero-shot et distilgpt2 ne produisent pas de vrais résumés (extraction brute ou hallucinations). Les 2000 exemples de SAMSum transforment radicalement le T5 en un résumeur de dialogues efficace. La comparaison encoder-decoder vs. decoder-only montre que l'architecture compte autant que la taille du modèle : malgré ses 82M de paramètres, distilgpt2 est très loin du T5-small (60M) sur cette tâche.

Nous retenons donc trois leçons :

- Fixer un protocole reproductible (seeds séparés pour les données et les modèles, mêmes splits, mêmes métriques) est essentiel pour que les comparaisons aient du sens.
- Combiner plusieurs métriques (ROUGE + EmbSim, micro + macro) révèle des phénomènes invisibles avec une seule métrique.
- Inclure des exemples qualitatifs est souvent plus informatif que les scores moyens : c'est en lisant les sorties qu'on détecte les hallucinations, les patterns de génération dégénérée, et les vrais progrès du fine-tuning.

5. Références

- [1] Lin, C.Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. Text Summarization Branches Out.
- [2] Raffel, C. et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. JMLR, 21(140), 1-67.
- [3] Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. NeurIPS Workshop.
- [4] Gliwa, B. et al. (2019). SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization. EMNLP Workshop.
- [5] Zhang, X. et al. (2015). Character-level Convolutional Networks for Text Classification. NeurIPS.
- [6] Pennington, J., Socher, R., Manning, C. (2014). GloVe: Global Vectors for Word Representation. EMNLP.
- [7] Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.
- [8] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS.
- [9] Holtzman, A. et al. (2020). The Curious Case of Neural Text Degeneration. ICLR.
- [10] Reimers, N., Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. EMNLP.